

# Lab Experiment 10

## SonarQube — Static Code Analysis

### Theory

#### Problem Statement

Code bugs and security issues are often found too late — during testing or even after deployment. Manual code reviews are slow, inconsistent, and don’t scale as teams grow. As software projects increase in size and complexity, the need for automated, consistent, and continuous code quality monitoring becomes critical.

#### What is SonarQube?

SonarQube is an open-source platform that automatically scans your source code for bugs, security vulnerabilities, and maintainability issues — without running the code. This process is known as Static Code Analysis (or Static Application Security Testing, SAST).

Unlike dynamic testing (which runs the application), static analysis examines the source code structure, patterns, and logic — catching problems at the earliest possible stage of development.

#### How SonarQube Solves the Problem

| Challenge              | SonarQube’s Solution                                              |
|------------------------|-------------------------------------------------------------------|
| Late bug discovery     | Scans on every commit, giving immediate feedback                  |
| Inconsistent reviews   | Applies a uniform, configurable rule set across all code          |
| No deployment safety   | Enforces Quality Gates — pass/fail checks before code is deployed |
| Growing technical debt | Tracks Technical Debt — estimates fix time for all flagged issues |
| Multi-language teams   | Supports 20+ programming languages                                |
| Lack of visibility     | Provides a visual dashboard for trends and history over time      |

#### Static Analysis vs Dynamic Analysis

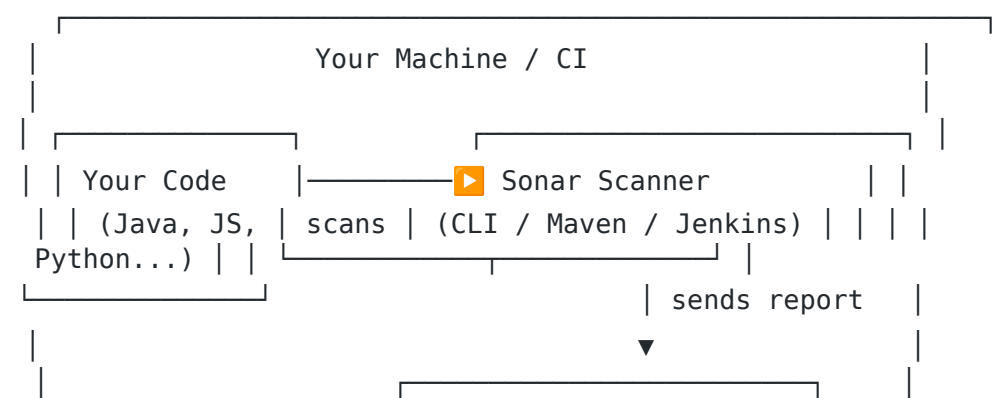
| Aspect             | Static Analysis (SonarQube) | Dynamic Analysis (Testing) |
|--------------------|-----------------------------|----------------------------|
| Code execution     | Not required                | Required                   |
| Stage              | Development / Pre-commit    | Testing / Runtime          |
| Speed              | Very fast                   | Slower                     |
| Coverage           | All code paths              | Only executed paths        |
| Security detection | Early                       | Late                       |

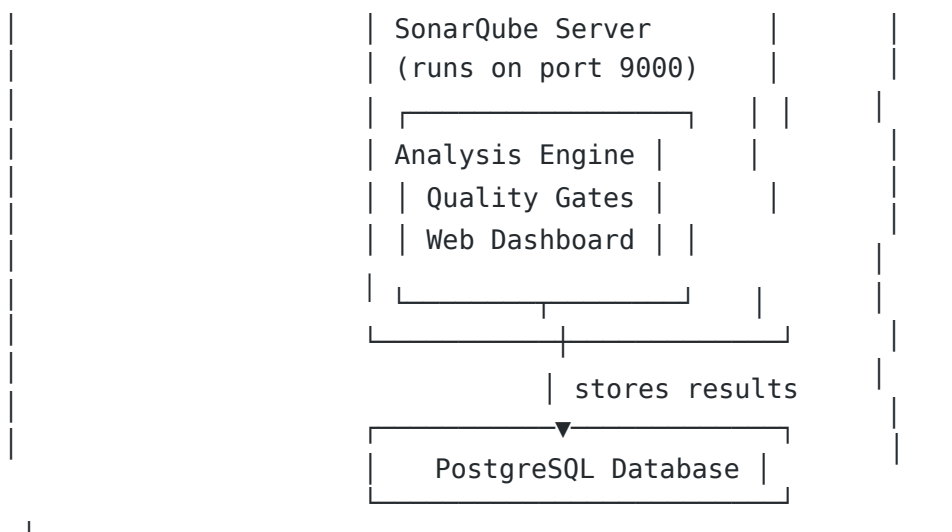
## Key Terminology

| Term           | Definition                                                   |
|----------------|--------------------------------------------------------------|
| Quality Gate   | A set of rules; code must pass before deployment             |
| Bug            | Code that will likely break or behave incorrectly at runtime |
| Vulnerability  | A security weakness in the code that could be exploited      |
| Code Smell     | Code that works but is poorly written or hard to maintain    |
| Technical Debt | Estimated time required to fix all identified issues         |
| Coverage       | Percentage of code tested by unit tests                      |
| Duplication    | Repeated code blocks (copy-paste code)                       |

## Lab Architecture

SonarQube has two separate components — a Server (the brain) and a Scanner (the worker). Both are required for the pipeline to function.





## Components: Server vs Scanner

### Part 1: SonarQube Server — “The Brain”

The SonarQube Server is a web application that:

- Stores all analysis results in a PostgreSQL database
- Applies code quality rules (bugs, vulnerabilities, smells)
- Shows a web dashboard at <http://localhost:9000>
- Enforces Quality Gates (pass/fail decisions)
- Tracks history and trends over time

Analogy: Think of it as a teacher / examiner — it receives work, grades it, and shows the report card.

Internal Components:

| Component           | Role                                            |
|---------------------|-------------------------------------------------|
| Analysis Engine     | Applies quality rules to submitted code reports |
| Web UI              | Dashboard you browse to at port 9000            |
| PostgreSQL Database | Persists all analysis results and history       |

### Part 2: Sonar Scanner — “The Worker”

The Sonar Scanner is a command-line tool that:

- Reads your source code files
- Detects issues based on the server’s rule set
- Sends an analysis report to the SonarQube Server

Analogy: Think of it as a student writing an exam — it does the work and submits it.

Scanner Types:

| Scanner Type                   | When to Use               |
|--------------------------------|---------------------------|
| sonar-scanner (CLI)            | Any project, run manually |
| Maven plugin (mvn sonar:sonar) | Java/Maven projects       |
| Gradle plugin                  | Java/Gradle projects      |
| Jenkins integration            | CI/CD pipelines           |
| GitHub Actions                 | Automated PRs             |

Why Both Are Required

Only Server installed

Dashboard is empty  
No code gets analyzed

Both installed together

Only Scanner installed

—  
Nowhere to send results  
Analysis is wasted

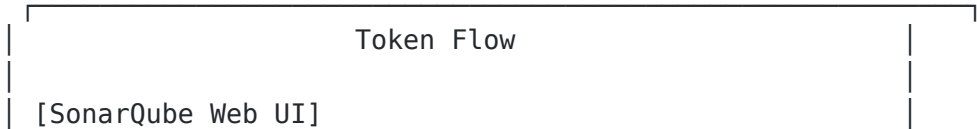
Full pipeline works ✓

Pipeline Flow:



Token Authentication Flow

The scanner does not use a username/password — it uses a token for authentication.



```

| Admin → My Account → Security → Generate Token
| Token: sqp_xxxxxxxx ◀ — copy this
|
|         |
|         | you paste it into:
|         ▼
| [Sonar Scanner]
| -Dsonar.login=sqp_xxxxxxxx
| OR
| export SONAR_TOKEN=sqp_xxxxxxxx
|
|         |
|         | HTTP request with token
|         ▼
| [SonarQube Server]
| Validates token → Accepts analysis → Stores results
|
|         |
|         ▼
| [Dashboard]
| http://localhost:9000 → view issues

```

### Token Rules:

- Generated once in the Server UI
- Used by the Scanner on every scan
- Shown only once — copy it immediately after generation

### Three Ways to Pass the Token:

```

# Option A – directly in the command sonar-
scanner -Dsonar.login=YOUR_TOKEN

# Option B – environment variable (preferred in
CI) export SONAR_TOKEN=YOUR_TOKEN sonar-scanner -
Dsonar.host.url=http://localhost:9000

# Option C – Maven flag mvn sonar:sonar
-Dsonar.login=YOUR_TOKEN

```

## Hands-On Lab Procedure

### Step 1: Start the SonarQube Server

Using Docker Compose to start SonarQube and its PostgreSQL database together:

```

# docker-compose.yml
version: '3.8'
services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar

```

```
POSTGRES_DB: sonarqube
POSTGRES_HOST_AUTH_METHOD: trust
volumes:
  - sonar-db-data:/var/lib/postgresql/data
```

```
networks:
  - sonarqube-lab
```

```
sonarqube:
  image: sonarqube:lts-community
  container_name: sonarqube
  ports:
    - "9000:9000"
  environment:
    SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
    SONAR_JDBC_USERNAME: sonar
    SONAR_JDBC_PASSWORD: sonar
  volumes:
    - sonar-data:/opt/sonarqube/data
    - sonar-extensions:/opt/sonarqube/extensions
  depends_on:
    - sonar-db
    networks:
    - sonarqube-lab
```

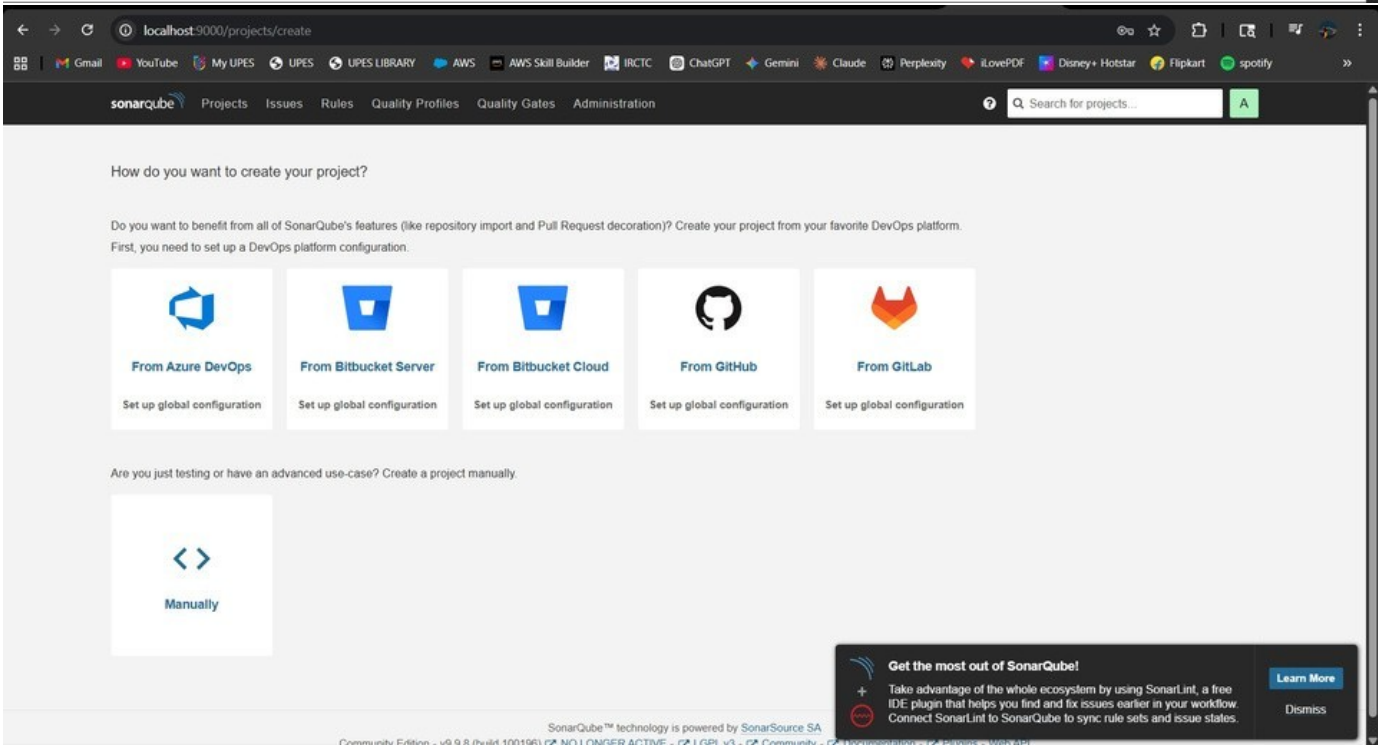
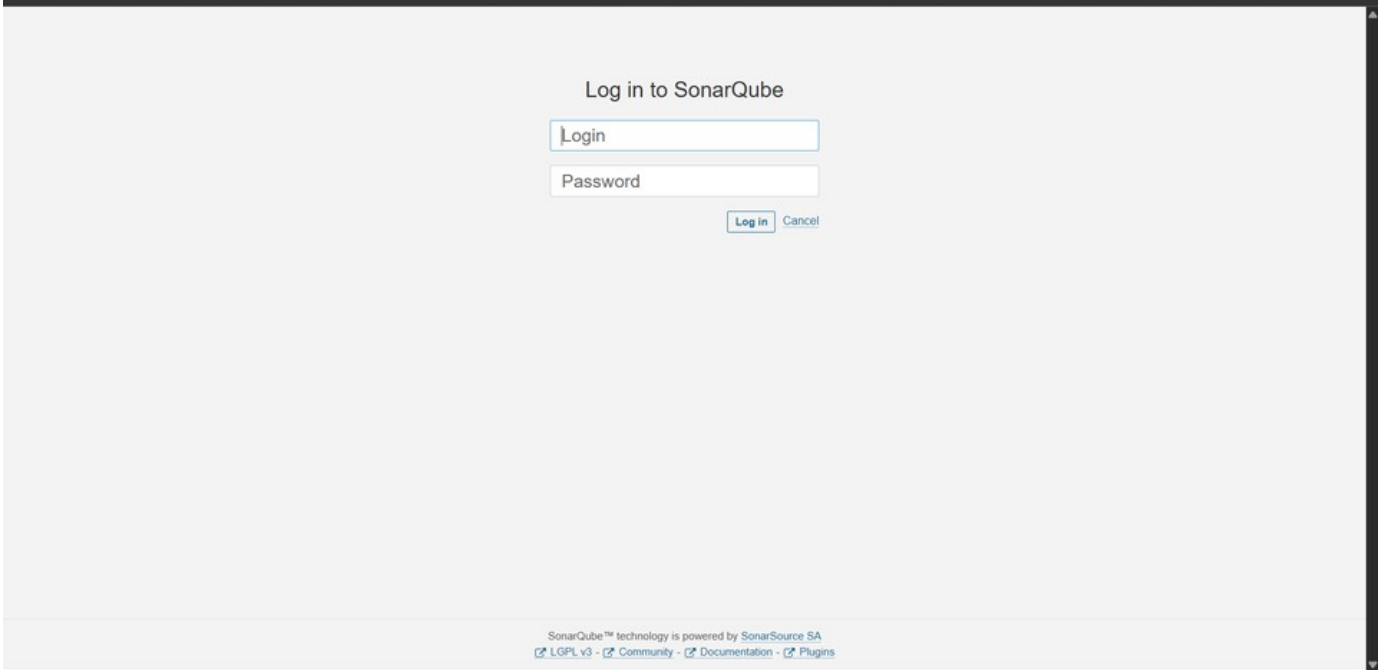
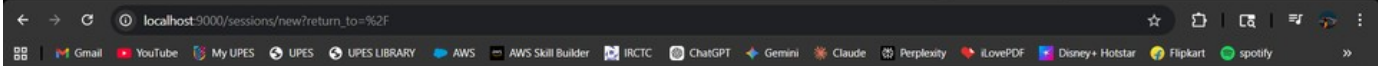
```
volumes:
  sonar-db-data: sonar-
data:
  sonar-extensions:
```

```
networks: sonarqube-
lab: driver:
bridge docker-compose
up -d
```

```
docker-compose logs -f sonarqube # wait for "SonarQube is up"
```

Access: <http://localhost:9000> — Default credentials: admin / admin

```
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'python' version : 3.24.1.11916 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'ruby' version : 1.11.0.3905 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'sonarscala' version : 1.11.0.3905 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'text' version : 2.0.2.1090 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'vbnets' version : 8.51.0.59060 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'web' version : 3.7.1.3306 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'xml' version : 2.7.0.3820 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'config' version : 1.2.0.267 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'csharp' version : 8.51.0.59060 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'flex' version : 2.8.0.3166 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'go' version : 1.11.0.3905 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'iac' version : 1.11.0.2847 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'jacoco' version : 1.3.0.1538 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'java' version : 7.16.0.30901 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'javascript' version : 9.13.0.20537 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'kotlin' version : 2.12.0.1056 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'php' version : 3.27.1.9352 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'python' version : 3.24.1.11916 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'ruby' version : 1.11.0.3905 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'sonarscala' version : 1.11.0.3905 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'text' version : 2.0.2.1090 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'vbnets' version : 8.51.0.59060 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'web' version : 3.7.1.3306 has not been found on the update center.
sonarqube | 2026.04.28 11:52:44 INFO web[AZ3T7rI+ds6SUsZgAAAW][o.s.u.c.UpdateCenter] The plugin 'xml' version : 2.7.0.3820 has not been found on the update center.
sonarqube | 2026.04.28 11:52:48 WARN es[[o.e.t.ThreadPool] absolute clock went backwards by [709ms/709ms] while timer thread was sleeping
sonarqube | 2026.04.28 11:53:19 WARN es[[o.e.t.ThreadPool] absolute clock went backwards by [809ms/809ms] while timer thread was sleeping
sonarqube | 2026.04.28 11:53:50 WARN es[[o.e.t.ThreadPool] absolute clock went backwards by [832ms/832ms] while timer thread was sleeping
sonarqube | 2026.04.28 11:54:21 WARN es[[o.e.t.ThreadPool] absolute clock went backwards by [837ms/837ms] while timer thread was sleeping
sonarqube | 2026.04.28 11:54:52 WARN es[[o.e.t.ThreadPool] absolute clock went backwards by [815ms/815ms] while timer thread was sleeping
sonarqube | 2026.04.28 11:55:38 WARN es[[o.e.t.ThreadPool] absolute clock went backwards by [1.2s/1269ms] while timer thread was sleeping
sonarqube | 2026.04.28 11:56:10 WARN es[[o.e.t.ThreadPool] absolute clock went backwards by [835ms/835ms] while timer thread was sleeping
^C
```



## Step 2: Create a Sample Java App with Intentional Code Issues

File: src/main/java/com/example/Calculator.java

```
package com.example;

public class Calculator {

    // BUG: Division by zero is not
    // handled public int divide(int a, int
    // b) { return a / b;
    // }

    // CODE SMELL: Unused variable
    public int add(int a, int b) {
        int result = a + b;
        int unused = 100; // flagged by SonarQube
        return result;
    }

    // VULNERABILITY: SQL Injection risk
    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id=" + userId;
        return query;
    }

    // CODE SMELL: Duplicated code public int multiply(int
    // a, int b) { int result = 0; for (int i = 0; i < b; i+
    // +) { result = result + a; } return result;
    // }

    public int multiplyAlt(int a, int b) { int result = 0; for (int i =
    // 0; i < b; i++) { result = result + a; } // exact duplicate return
    // result;
    // }

    // BUG: Null pointer risk public String getName(String name)
    // { return name.toUpperCase(); // NullPointerException if name is
    // null }

    // CODESMELL: Empty catch block
    public void riskyOperation()
    { try {
        int x = 10 / 0;
    } catch (Exception e) {
        // never leave catch blocks empty
    }
    }
}
```



```

sample-java-app > src > main > java > com > example > Calculator.java
1  package com.example;
2
3  public class Calculator {
4
5      public int divide(int a, int b) {
6          return a / b; // bug
7      }
8
9      public int add(int a, int b) {
10         int result = a + b;
11         int unused = 100; // code smell
12         return result;
13     }
14
15     public String getUser(String userId) {
16         String query = "SELECT * FROM users WHERE id=" + userId; // vulnerability
17         return query;
18     }
19
20     public int multiply(int a, int b) {
21         int result = 0;
22         for(int i=0;i<b;i++){
23             result += a;
24         }
25         return result;
26     }
27
28     public int multiplyAlt(int a, int b) {
29         int result = 0;
30         for(int i=0;i<b;i++){
31             result += a;
32         }
33         return result;
34     }
35
36     public String getName(String name) {
37         return name.toUpperCase(); // null risk
38     }
39
40     public void riskyOperation() {
41         try {
42             int x = 10 / 0;
43         } catch (Exception e) {
44         }
45     }
46 }
47

```

## Summary of Intentional Issues Introduced:

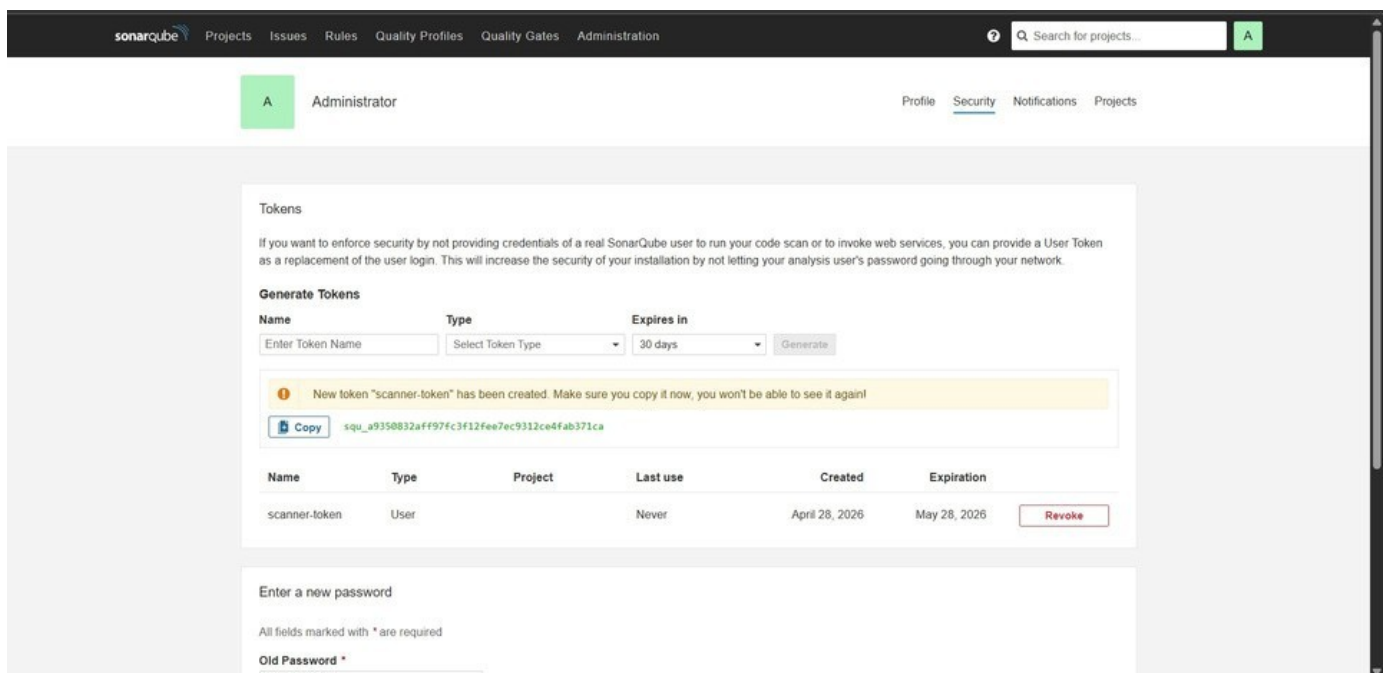
| Issue Type    | Description                            | Line/Method                |
|---------------|----------------------------------------|----------------------------|
| Bug           | Division by zero not handled           | divide()                   |
| Bug           | Null pointer risk                      | getName()                  |
| Vulnerability | SQL Injection via string concatenation | getUser()                  |
| Code Smell    | Unused variable                        | add()                      |
| Code Smell    | Duplicated code blocks                 | multiply() / multiplyAlt() |

| Issue Type | Description       | Line/Method      |
|------------|-------------------|------------------|
| Code Smell | Empty catch block | riskyOperation() |

## Step 3: Generate Authentication Token

1. Open `http://localhost:9000`
2. Log in as admin
3. Click user icon (top-right) → My Account
4. Click the Security tab
5. Under “Generate Tokens” → type name: `scanner-token`
6. Click Generate
7. Copy the token immediately — it looks like: `sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX`

 The token is shown only once. Store it securely.



## Step 4: Run the Scanner

Option A — Maven Plugin (recommended for Java):

```
cd sample-java-app mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

```
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.pom (12 kB at 14 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/parent/parent/59.0.29/parent-59.0.29.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/parent/parent/59.0.29/parent-59.0.29.pom (32 kB at 674 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.jar (50 kB at 1.1 MB/s)
[INFO] -----[ jar ]-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- sonar:3.9.1.2184:sonar (default-cli) @ sample-app ---
Downloading from central: https://repo.maven.apache.org/maven2/org/sonatype/plexus/plexus-sec-dispatcher/1.4/plexus-sec-dispatcher-1.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonatype/plexus/plexus-sec-dispatcher/1.4/plexus-sec-dispatcher-1.4.pom (3.0 kB at 63 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/sonatype/spice/spice-parent/12/spice-parent-12.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonatype/spice/spice-parent/12/spice-parent-12.pom (6.8 kB at 139 kB/s)
```

Run via Docker:

```
docker run --rm \
network sonarqube-lab \
-e SONAR_TOKEN="YOUR_TOKEN" \
-v "$(pwd):/usr/src" \
sonarsource/sonar-scanner-cli \
-Dsonar.host.url=http://sonarqube:9000 \
-Dsonar.projectBaseDir=/usr/src \
-Dsonar.projectKey=sample-java-app

--network lab10_sonarqube-lab \
-e SONAR_TOKEN="squ_a9350832aff97fc3f12fee7ec9312ce4fab371ca" \
-v "$(pwd):/usr/src" \
sonarsource/sonar-scanner-cli \
-Dsonar.host.url=http://sonarqube:9000 \
-Dsonar.projectBaseDir=/usr/src \
-Dsonar.projectKey=sample-java-app \
-Dsonar.java.binaries=-
Unable to find image 'sonarsource/sonar-scanner-cli:latest' locally
latest: Pulling from sonarsource/sonar-scanner-cli
fd916ef360dd: Pull complete
b226ade5bac4: Pull complete
677b33267ef4: Pull complete
f95d770a13e7: Pull complete
4f4fb700ef54: Pull complete
8f515facfbc5: Pull complete
Digest: sha256:a429d20d22c8d97b9a10ea5585f1c56d2a54796f0ae3da06c7a1a49eec78e6c2
Status: Downloaded newer image for sonarsource/sonar-scanner-cli:latest
12:24:43.771 INFO Scanner configuration file: /opt/sonar-scanner/conf/sonar-scanner.properties
12:24:43.776 INFO Project root configuration file: NONE
```

Docker Flag Reference:

| Flag | Purpose                          |
|------|----------------------------------|
| --rm | Auto-delete container after scan |

| Flag                   | Purpose                                 |
|------------------------|-----------------------------------------|
| - -network             | Connect to SonarQube Docker network     |
| -e SONAR_TOKEN         | Authentication token                    |
| -v                     | Mount source code into container        |
| -Dsonar.host.url       | Server URL (use container name, not IP) |
| -Dsonar.projectBaseDir | Project root inside container           |
| -Dsonar.projectKey     | Project identifier on the server        |

## Step 5: View Results in the Dashboard

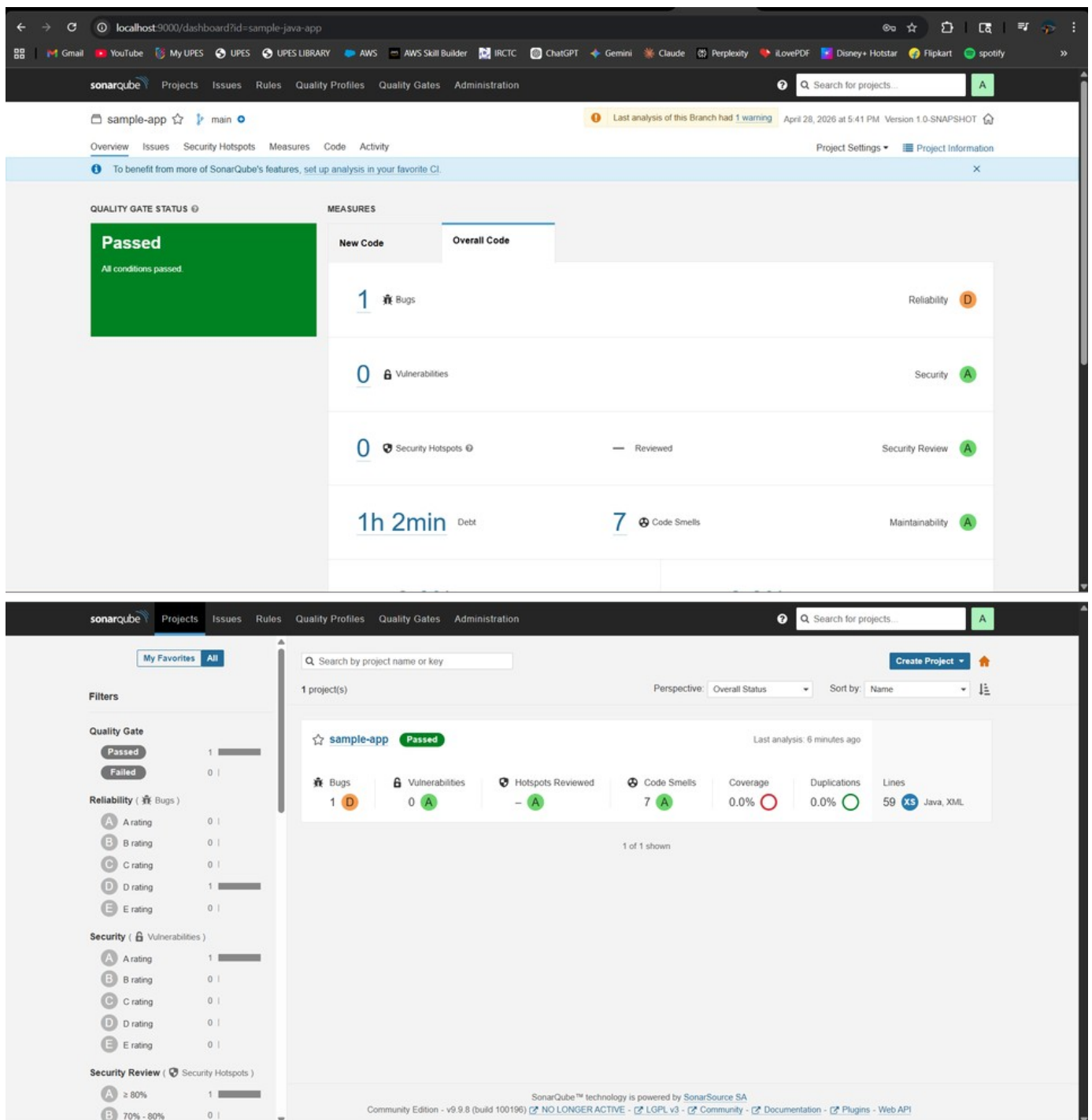
Access: <http://localhost:9000/dashboard?id=sample-java-app>

Expected Dashboard Output:

| sample-java-app – Dashboard         |           |             |
|-------------------------------------|-----------|-------------|
| Bugs                                | Vulnerab. | Code Smells |
| 5                                   | 1         | 8           |
| Coverage: 0% Duplications: 2 blocks |           |             |
| Technical Debt: ~1h 30min           |           |             |
| Quality Gate: ✗ FAILED              |           |             |

You can also query results via the REST API:

```
curl -u admin:YOUR_TOKEN \  
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=BUG"
```



## Maven Scanner Execution — Key Events

The scanner log (starting around line 884 in text.txt) shows the complete Maven scan run:

| Phase                    | Detail                                            |
|--------------------------|---------------------------------------------------|
| Project loading          | sample-app loaded, analysis cache hit (116 bytes) |
| Files indexed            | 2 files indexed                                   |
| Quality profiles applied | Java: Sonar way, XML: Sonar way                   |

| Phase                      | Detail                                                                             |
|----------------------------|------------------------------------------------------------------------------------|
| Java sensor                | Analyzed 1 Main Java source file (Calculator.java) using ECJ batch — 100% analyzed |
| JaCoCo                     | No coverage report found (expected — no tests written) → Coverage = 0%             |
| CSS/HTML/VueJS             | No such files found — skipped                                                      |
| XML sensor                 | 1 source file analyzed (pom.xml)                                                   |
| Text/Secrets sensor        | 2 source files analyzed                                                            |
| CPD (Copy-Paste Detection) | Calculated for 1 file                                                              |

### Final Build Outcome:

```
[INFO] ANALYSIS SUCCESSFUL
[INFO] Results at: http://localhost:9000/dashboard?id=sample-java-app
[INFO] BUILD SUCCESS
[INFO] Total time: 9.790 s
[INFO] Finished at: 2026-04-28T18:11:26+05:30
```

The analysis completed in 9.79 seconds total (6.1s for analysis itself), and the report was successfully uploaded to the SonarQube server.

## Jenkins CI/CD Integration

---

Once SonarQube is working locally, it can be integrated into a Jenkins pipeline so every commit is automatically scanned.

```
// Jenkinsfile
pipeline {
    agent any
    environment {
        SONAR_HOST_URL = 'http://sonarqube:9000'
        SONAR_TOKEN = credentials('sonar-token')
    }
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('SonarQube') {
```



```

        sh 'mvn clean verify sonar:sonar'
    }
    } } stage('Quality Gate') { steps {
timeout(time: 5, unit: 'MINUTES') {
waitForQualityGate abortPipeline: true
}
} } stage('Build') { steps
{ sh 'mvn package' }
} stage('Deploy') { steps { sh 'docker build -t
sample-app .' sh 'docker run -d -p 8080:8080
sample-app'
}
}
}
}
}
}

```

```

[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.pom (27 kB at 49 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.pom (766 B at 23 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-parent/1.3/hamcrest-parent-1.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-parent/1.3/hamcrest-parent-1.3.pom (2.0 kB at 64 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.jar (385 kB at 3.2 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.jar (45 kB at 1.2 MB/s)
[INFO]

```

```
~$ docker inspect sonarqube | grep -i network
```

```

"NetworkMode": "lab10_sonarqube-lab",
"NetworkSettings": {
  "Networks": {
    "NetworkID": "be71b00fed29e4b362d66768a543f6433fd2b312f976d10587021e88dd748d2f",

```

Pipeline Flow:

Checkout → SonarQube Scan → Quality Gate Check → Build → Deploy

FAIL?

Pipeline stops.

Code does not get deployed.

## Comparative Analysis

# DevOps Tool Comparison Matrix

| Feature             | Jenkins             | Ansible                | Chef              | SonarQube       |
|---------------------|---------------------|------------------------|-------------------|-----------------|
| Primary Purpose     | CI/CD Automation    | Config Management      | Config Management | Code Quality    |
| Architecture        | Master-Agent        | Agentless              | Client-Server     | Client-Server   |
| Language            | Java / Groovy       | YAML                   | Ruby              |                 |
| Learning Curve      | Moderate            | Low                    | High              | Java Low        |
| Setup               | Moderate            | Simple                 | Complex           | Simple          |
| Complexity Use Case | Build, Test, Deploy | Infrastructure as Code | Enterprise CM     | Static Analysis |

## Key Differences

Jenkins vs. Configuration Management: Use Jenkins to orchestrate pipelines (build → test → deploy). Use Ansible or Chef to maintain server configuration. In practice, Jenkins calls Ansible/Chef as part of the deployment step.

Ansible vs. Chef:

| Aspect         | Ansible            | Chef              |
|----------------|--------------------|-------------------|
| Complexity     | Simple, YAML       | Complex, Ruby     |
| Agent required | No (agentless)     | Yes (chef-client) |
| Best for       | Small-medium teams | Large enterprise  |

SonarQube in CI/CD: Scan on every pull request and every nightly build. Use Quality Gates to block deployments when code quality drops below the configured threshold.

## Summary

| Concept | Description | |———|———| | SonarQube | The analysis platform that stores and displays results | | Sonar Scanner | The tool that reads your code and sends the report | | Both Required | Scanner without Server = nowhere to send; Server without Scanner = nothing to analyze | | Token | Secure authentication key generated once in the Web UI; used by Scanner on every run | | Quality Gate | Pass/fail threshold that blocks bad code from deployment | | Technical Debt | Quantified estimate of the effort needed to fix all flagged issues | —